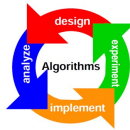


Estrategias algorítmicas

José A. Rodríguez Melquiades

31 de mayo de 2026

Algoritmos recursivos



- 1 Recursividad
- 2 Algoritmo recursivo
 - Reparto de herencia
- 3 Problemas de naturaleza recursiva

Recursividad

- **Sucesión:** Colección enumerada de objetos, es decir, una estructura contable que representa una lista ordenada de elementos o números.

$$\{a_n\} = \{a_0, a_1, \dots, a_n\}, \forall n \in \mathbb{Z}^+$$

Recursividad

- **Sucesión:** Colección enumerada de objetos, es decir, una estructura contable que representa una lista ordenada de elementos o números.

$$\{a_n\} = \{a_0, a_1, \dots, a_n\}, \forall n \in \mathbb{Z}^+$$

- ▶ **Enfoque matemático:** Lista ordenada de elementos, generalmente números, donde cada elemento ocupa una posición determinada.

$$a_n = \frac{1}{n}; \lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

Recursividad

- **Sucesión:** Colección enumerada de objetos, es decir, una estructura contable que representa una lista ordenada de elementos o números.

$$\{a_n\} = \{a_0, a_1, \dots, a_n\}, \forall n \in \mathbb{Z}^+$$

- ▶ **Enfoque matemático:** Lista ordenada de elementos, generalmente números, donde cada elemento ocupa una posición determinada.

$$a_n = \frac{1}{n}; \lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

- ▶ **Enfoque informático:** Se interpreta como estructura ordenada de datos o como una secuencia generada por un algoritmo.

Recursividad

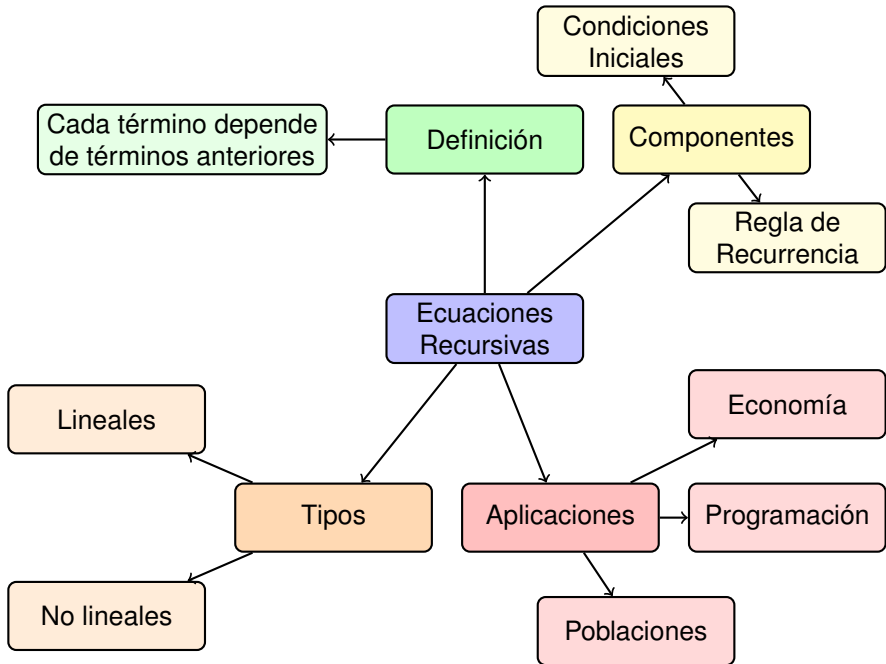
- **Sucesión:** Colección enumerada de objetos, es decir, una estructura contable que representa una lista ordenada de elementos o números.

$$\{a_n\} = \{a_0, a_1, \dots, a_n\}, \forall n \in \mathbb{Z}^+$$

- ▶ **Enfoque matemático:** Lista ordenada de elementos, generalmente números, donde cada elemento ocupa una posición determinada.

$$a_n = \frac{1}{n}; \lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

- ▶ **Enfoque informático:** Se interpreta como estructura ordenada de datos o como una secuencia generada por un algoritmo.
 - ★ Arreglos (Arrays); Vectores; Listas enlazadas; Cadenas de caracteres; sucesiones para analizar algoritmos.



- Ecuación recursiva: Dos partes:

- **Ecuación recursiva:** Dos partes:

- ▶ **Condición inicial:** Uno o mas términos de la sucesión que precede al primer término, es decir, es el inicio de la recurrencia.

$$a_0 = 3$$

- **Ecuación recursiva:** Dos partes:

- ▶ **Condición inicial:** Uno o mas términos de la sucesión que precede al primer término, es decir, es el inicio de la recurrencia.

$$a_0 = 3$$

- ▶ **Relación de recurrencia:** Ecuación que usa una regla para generar el siguiente término, es decir, aquella definida en términos de si mismo.

$$a_n = 5a_{n-1} + 3$$

- Ejemplos:

- Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$

- Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$

- ▶ $a_n = a_{n-1} + 3, a_0 = 2 \forall n \geq 1$

- Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$
- ▶ $a_n = a_{n-1} + 3, a_0 = 2 \forall n \geq 1$
- ▶ $a_n = 2a_{n-1}, \forall n \geq 1$

● Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$
- ▶ $a_n = a_{n-1} + 3, a_0 = 2 \forall n \geq 1$
- ▶ $a_n = 2a_{n-1}, \forall n \geq 1$
- ▶ $F_n = F_{n-1} + F_{n-2}, \forall n \geq 2; F_0 = 0; F_1 = 1$

● Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$
- ▶ $a_n = a_{n-1} + 3, a_0 = 2 \forall n \geq 1$
- ▶ $a_n = 2a_{n-1}, \forall n \geq 1$
- ▶ $F_n = F_{n-1} + F_{n-2}, \forall n \geq 2; F_0 = 0; F_1 = 1$
- ▶ $a_n = 3a_{n-1} - 2a_{n-2}; a_0 = 1; a_1 = 2$

● Ejemplos:

- ▶ $a_n = a_{n-1} + d, \forall d \in \mathbb{N}$
- ▶ $a_n = a_{n-1} + 3, a_0 = 2 \forall n \geq 1$
- ▶ $a_n = 2a_{n-1}, \forall n \geq 1$
- ▶ $F_n = F_{n-1} + F_{n-2}, \forall n \geq 2; F_0 = 0; F_1 = 1$
- ▶ $a_n = 3a_{n-1} - 2a_{n-2}; a_0 = 1; a_1 = 2$
- ▶ $x_n = x_{n-1}^2 - 1; x_0 = 2$

- Ejemplos: $a_n \equiv T(n)$

- Ejemplos: $a_n \equiv T(n)$



$$T(n) = \begin{cases} 3T(n-1) - 15 & \text{si } n \geq 2 \\ 8 & \text{si } n = 1 \end{cases}$$

- Ejemplos: $a_n \equiv T(n)$



$$T(n) = \begin{cases} 3T(n-1) - 15 & \text{si } n \geq 2 \\ 8 & \text{si } n = 1 \end{cases}$$



$$T(n) = \begin{cases} T(n-1) + n - 1 & \text{si } n \geq 2 \\ 2 & \text{si } n = 1 \end{cases}$$

- Uso de la ecuación recursiva:

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.
- ▶ **Series numéricas:** Números de Fibonacci.

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.
- ▶ **Series numéricas:** Números de Fibonacci.
- ▶ **Economía:** Desarrollar modelos discretos de crecimiento.

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.
- ▶ **Series numéricas:** Números de Fibonacci.
- ▶ **Economía:** Desarrollar modelos discretos de crecimiento.
 - ★ Crecimiento poblacional: $P_n = 1,05P_{n-1}$ con $P_0 = 1000$

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.
- ▶ **Series numéricas:** Números de Fibonacci.
- ▶ **Economía:** Desarrollar modelos discretos de crecimiento.
 - ★ Crecimiento poblacional: $P_n = 1,05P_{n-1}$ con $P_0 = 1000$
- ▶ **Informática:** En estructura de datos, análisis de algoritmos.

- **Uso de la ecuación recursiva:**

- ▶ **Algoritmos recursivos:** Determinar la complejidad y el tiempo.
- ▶ **Combinatoria:** Binomios, particiones.
- ▶ **Series numéricas:** Números de Fibonacci.
- ▶ **Economía:** Desarrollar modelos discretos de crecimiento.
 - ★ Crecimiento poblacional: $P_n = 1,05P_{n-1}$ con $P_0 = 1000$
- ▶ **Informática:** En estructura de datos, análisis de algoritmos.
- ▶ Procesos físicos o biológicos discretos.

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(1) = 2T(0) + 1 = 2(1) + 1 = 3$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(1) = 2T(0) + 1 = 2(1) + 1 = 3$
- ▶ $T(2) = 2T(1) + 1 = 2(3) + 1 = 7$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(1) = 2T(0) + 1 = 2(1) + 1 = 3$
- ▶ $T(2) = 2T(1) + 1 = 2(3) + 1 = 7$
- ▶ $T(3) = 2T(2) + 1 = 2(7) + 1 = 15$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(1) = 2T(0) + 1 = 2(1) + 1 = 3$
- ▶ $T(2) = 2T(1) + 1 = 2(3) + 1 = 7$
- ▶ $T(3) = 2T(2) + 1 = 2(7) + 1 = 15$
- ▶ $T(4) = 2T(3) + 1 = 2(15) + 1 = 31$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 2T(n-1) + 1 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(1) = 2T(0) + 1 = 2(1) + 1 = 3$
- ▶ $T(2) = 2T(1) + 1 = 2(3) + 1 = 7$
- ▶ $T(3) = 2T(2) + 1 = 2(7) + 1 = 15$
- ▶ $T(4) = 2T(3) + 1 = 2(15) + 1 = 31$
- ▶ Por lo tanto: $T(n) = 2^{n+1} - 1$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

► $T(0) = 1$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(0) = 1$
- ▶ $T(1) = 3T(0) + 2 = 3(1) + 2 = 5$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(0) = 1$
- ▶ $T(1) = 3T(0) + 2 = 3(1) + 2 = 5$
- ▶ $T(2) = 3T(1) + 2 = 3(5) + 2 = 17$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(0) = 1$
- ▶ $T(1) = 3T(0) + 2 = 3(1) + 2 = 5$
- ▶ $T(2) = 3T(1) + 2 = 3(5) + 2 = 17$
- ▶ $T(3) = 3T(2) + 2 = 3(17) + 2 = 53$

- **Ejemplo:** Resolver la ecuación recursiva sgte.

$$T(n) = \begin{cases} 3T(n-1) + 2 & \text{si } n \geq 1 \\ 1 & \text{si } n = 0 \end{cases}$$

- ▶ $T(0) = 1$
- ▶ $T(1) = 3T(0) + 2 = 3(1) + 2 = 5$
- ▶ $T(2) = 3T(1) + 2 = 3(5) + 2 = 17$
- ▶ $T(3) = 3T(2) + 2 = 3(17) + 2 = 53$
- ▶ $T(4) = 3T(3) + 2 = 3(53) + 2 = 161$
- ▶ Por lo tanto en general asumir que: $T(n) = M \times 3^n + N$

- Ejemplo: (cont.)

- Ejemplo: (cont.)

- ▶ Para $n = 0$: $T(0) = M \times 3^0 + N = M + N \implies M + N = 1$

- Ejemplo: (cont.)

- ▶ Para $n = 0$: $T(0) = M \times 3^0 + N = M + N \implies M + N = 1$

- ▶ Para $n = 1$: $T(1) = M \times 3^1 + N = 3M + N \implies 3M + N = 5$

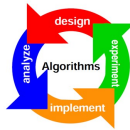
- Ejemplo: (cont.)

- ▶ Para $n = 0$: $T(0) = M \times 3^0 + N = M + N \implies M + N = 1$
- ▶ Para $n = 1$: $T(1) = M \times 3^1 + N = 3M + N \implies 3M + N = 5$
- ▶ Resolviendo el sistema de ecuaciones: $M = 2$; $N = -1$

- Ejemplo: (cont.)

- ▶ Para $n = 0$: $T(0) = M \times 3^0 + N = M + N \implies M + N = 1$
- ▶ Para $n = 1$: $T(1) = M \times 3^1 + N = 3M + N \implies 3M + N = 5$
- ▶ Resolviendo el sistema de ecuaciones: $M = 2$; $N = -1$
- ▶ Por lo tanto, $T(n) = 2 \times 3^n - 1$

Algoritmos recursivos



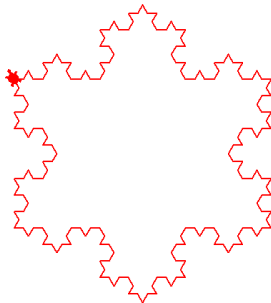
- 1 Recursividad
- 2 Algoritmo recursivo
 - Reparto de herencia
- 3 Problemas de naturaleza recursiva

Algoritmo recursivo

- Aquel algoritmo cuando dentro de su cuerpo o contenido, de forma directa o indirecta se realiza una llamada a él mismo.

Algoritmo recursivo

- Aquel algoritmo cuando dentro de su cuerpo o contenido, de forma directa o indirecta se realiza una llamada a él mismo.
- Obtiene la solución del problema a través de una llamada a sí mismo. Esta llamada se denomina recursiva o recurrente, donde cada llamada se define sobre un problema de menor tamaño.



- Característica formal:

- **Característica formal:**

- ▶ **Caso base:** Condición que detiene el proceso de la recursión.

- **Característica formal:**

- ▶ **Caso base:** Condición que detiene el proceso de la recursión.
- ▶ **Llamada recursiva:** El problema se resuelve llamando a la misma función en momentos mas simple del problema.

- **Característica formal:**

- ▶ **Caso base:** Condición que detiene el proceso de la recursión.
- ▶ **Llamada recursiva:** El problema se resuelve llamando a la misma función en momentos mas simple del problema.

- **Observación 1:** El proceso determina la descomposición, es decir que el problema se divide hasta llegar a los casos base.

- **Característica formal:**

- ▶ **Caso base:** Condición que detiene el proceso de la recursión.
- ▶ **Llamada recursiva:** El problema se resuelve llamando a la misma función en momentos mas simple del problema.

- **Observación 1:** El proceso determina la descomposición, es decir que el problema se divide hasta llegar a los casos base.

- **Observación 2:** Todo algoritmo recursivo debe incluir al menos un caso base, para garantizar que se ejecutará en algún momento evitando la recursividad infinita.

- **Ejemplo:** Factorial de un número

1: Función Fact(n)

2: **if** $n = 1$ **then** Retornar 1

3: **else** $n * \text{Fact}(n - 1)$

4: **end if**

▷ Caso base

▷ Caso recursivo

Si $n = 1$ retornar 1: Condición de **salida o caso base** evita que la función se llame a sí misma indefinidamente.

- **Ejemplo:** Factorial de un número

```
1: Función Fact(n)
2: if  $n = 1$  then Retornar 1
3: else  $n * \text{Fact}(n - 1)$ 
4: end if
```

▷ Caso base
▷ Caso recursivo

Si $n = 1$ retornar 1: Condición de **salida o caso base** evita que la función se llame a sí misma indefinidamente.

- **Ejemplo:** Suma recursiva de los elementos de un arreglo

```
1: function SUMAARREGLO(A, n)
2:   if  $n = 0$  then
3:     retornar 0
4:   else
5:     retornar  $A[n] + \text{SUMAARREGLO}(A, n - 1)$ 
6:   end if
7: end function
```

▷ Caso base
▷ Caso recursivo

- Ejemplo: (cont.)

$$S(n) = \begin{cases} 0, & \text{si } n = 0 \\ A[n] + S(n-1), & \text{si } n > 0 \end{cases}$$

donde:

- ▶ $S(n)$: Suma de los n elementos del arreglo.
- ▶ $A[n]$: Elemento en la posición n .

Test:

- Ejemplo: (cont.)

$$S(n) = \begin{cases} 0, & \text{si } n = 0 \\ A[n] + S(n-1), & \text{si } n > 0 \end{cases}$$

donde:

- ▶ $S(n)$: Suma de los n elementos del arreglo.
- ▶ $A[n]$: Elemento en la posición n .

Test:

- ▶ Arreglo: $A = [4, 7, 2, 5]$, donde $n = 4$.

- Ejemplo: (cont.)

$$S(n) = \begin{cases} 0, & \text{si } n = 0 \\ A[n] + S(n-1), & \text{si } n > 0 \end{cases}$$

donde:

- ▶ $S(n)$: Suma de los n elementos del arreglo.
- ▶ $A[n]$: Elemento en la posición n .

Test:

- ▶ Arreglo: $A = [4, 7, 2, 5]$, donde $n = 4$.
- ▶ Llamada inicial: $\text{SumaArreglo}(A, 4)$.

- Ejemplo: (cont.)

$$S(n) = \begin{cases} 0, & \text{si } n = 0 \\ A[n] + S(n-1), & \text{si } n > 0 \end{cases}$$

donde:

- ▶ $S(n)$: Suma de los n elementos del arreglo.
- ▶ $A[n]$: Elemento en la posición n .

Test:

- ▶ Arreglo: $A = [4, 7, 2, 5]$, donde $n = 4$.
- ▶ Llamada inicial: $\text{SumaArreglo}(A, 4)$.
- ▶ Llamadas recursivas: $4 + 7 + 2 + 5 + 0$

- **Ejemplo:** Conteo recursivo de dígitos

1: **function** CONTARDIGITOS(n)

2: **if** $n < 10$ **then**

3: **retornar** 1

▷ Caso base

4: **else**

5: **retornar** $1 + \text{CONTARDIGITOS}(\lfloor n/10 \rfloor)$

▷ Caso recursivo

6: **end if**

7: **end function**

$$D(n) = \begin{cases} 1, & \text{si } n < 10 \\ 1 + D\left(\left\lfloor \frac{n}{10} \right\rfloor\right), & \text{si } n \geq 10 \end{cases}$$

Test:

- **Ejemplo:** Conteo recursivo de dígitos

1: **function** CONTARDIGITOS(n)

2: **if** $n < 10$ **then**

3: **retornar** 1

▷ Caso base

4: **else**

5: **retornar** $1 + \text{CONTARDIGITOS}(\lfloor n/10 \rfloor)$

▷ Caso recursivo

6: **end if**

7: **end function**

$$D(n) = \begin{cases} 1, & \text{si } n < 10 \\ 1 + D\left(\left\lfloor \frac{n}{10} \right\rfloor\right), & \text{si } n \geq 10 \end{cases}$$

Test:

▶ $\text{ContarDigitos}(5482) = 1 + \text{ContarDigitos}(548) = 1 + 1 + \text{ContarDigitos}(54)$

- **Ejemplo:** Conteo recursivo de dígitos

1: **function** CONTARDIGITOS(n)

2: **if** $n < 10$ **then**

3: **retornar** 1

▷ Caso base

4: **else**

5: **retornar** $1 + \text{CONTARDIGITOS}(\lfloor n/10 \rfloor)$

▷ Caso recursivo

6: **end if**

7: **end function**

$$D(n) = \begin{cases} 1, & \text{si } n < 10 \\ 1 + D\left(\left\lfloor \frac{n}{10} \right\rfloor\right), & \text{si } n \geq 10 \end{cases}$$

Test:

- ▶ $\text{ContarDigitos}(5482) = 1 + \text{ContarDigitos}(548) = 1 + 1 + \text{ContarDigitos}(54)$
- ▶ $= 1 + 1 + 1 + \text{ContarDigitos}(5) = 1 + 1 + 1 + 1 = 4$

Reparto de herencia

- **Problema:** Se desea calcular la cantidad total de herencia que una persona recibe no solo de sus padres, sino también de todos sus antepasados (abuelos, bisabuelos, etc.), considerando que cada generación hereda un porcentaje (por ejemplo, la mitad) de la herencia de sus padres.

Reparto de herencia

- **Problema:** Se desea calcular la cantidad total de herencia que una persona recibe no solo de sus padres, sino también de todos sus antepasados (abuelos, bisabuelos, etc.), considerando que cada generación hereda un porcentaje (por ejemplo, la mitad) de la herencia de sus padres.
- **Proceso:**

Reparto de herencia

- **Problema:** Se desea calcular la cantidad total de herencia que una persona recibe no solo de sus padres, sino también de todos sus antepasados (abuelos, bisabuelos, etc.), considerando que cada generación hereda un porcentaje (por ejemplo, la mitad) de la herencia de sus padres.
- **Proceso:**
 - ▶ **Entrada:** Familia; HerenciaDirecta.

Reparto de herencia

- **Problema:** Se desea calcular la cantidad total de herencia que una persona recibe no solo de sus padres, sino también de todos sus antepasados (abuelos, bisabuelos, etc.), considerando que cada generación hereda un porcentaje (por ejemplo, la mitad) de la herencia de sus padres.
- **Proceso:**
 - ▶ **Entrada:** Familia; HerenciaDirecta.
 - ▶ **Salida:** HerenciaTotal.

● Seudocódigo:

```
1: function HERENCIATOTAL(persona)
2:   if persona ya está en HerenciaMemoria then
3:     Retornar HerenciaMemoria(persona)
4:   end if
5:   if persona NO tiene padres then
6:     valor := HerenciaDirecta(persona) SI existe, SINO 0
7:   else
8:     valor := 0
9:     for cada padre en Familia(persona) do
10:      valor := valor + 0.5 * HerenciaTotal(padre)
11:    end for
12:   end if
13:   HerenciaMemoria(persona) := valor
14:   Retornar valor
15: end function
16: Inicializar HerenciaMemoria := vacío
17: Obtener el conjunto de todas las personas de Familia
18: for cada persona en el conjunto total do
19:   HerenciaTotal(persona)
20: end for
21: for cada persona ORDENADA en HerenciaMemoria do
22:   Retornar persona, HerenciaMemoria[persona]
23: end for
```

- Ecuación recursiva:

- Ecuación recursiva:
 - ▶ Datos:

- Ecuación recursiva:

- ▶ Datos:

- ★ $H(p)$: Herencia total que recibe la persona p .

- Ecuación recursiva:

- ▶ Datos:

- ★ $H(p)$: Herencia total que recibe la persona p .
 - ★ $h(p)$: Herencia Directa (propia) de la persona p (siempre que este en el diccionario llamado Herencia).

● Ecuación recursiva:

► Datos:

- ★ $H(p)$: Herencia total que recibe la persona p .
- ★ $h(p)$: Herencia Directa (propia) de la persona p (siempre que este en el diccionario llamado Herencia).
- ★ $P(p)$: Conjunto de padres de la persona p .

● Ecuación recursiva:

► Datos:

- ★ $H(p)$: Herencia total que recibe la persona p .
- ★ $h(p)$: Herencia Directa (propia) de la persona p (siempre que este en el diccionario llamado Herencia).
- ★ $P(p)$: Conjunto de padres de la persona p .
- ★ r : Porcentaje de herencia recibido de cada padre (por defecto 0.5)

● Ecuación recursiva:

► Datos:

- ★ $H(p)$: Herencia total que recibe la persona p .
- ★ $h(p)$: Herencia Directa (propia) de la persona p (siempre que este en el diccionario llamado Herencia).
- ★ $P(p)$: Conjunto de padres de la persona p .
- ★ r : Porcentaje de herencia recibido de cada padre (por defecto 0.5)

► Ecuación recursiva:

$$H(p) = h(p) + r \times \sum_{q \in P(p)} H(q) \quad (1)$$

- Ejemplo:

- Ejemplo:
 - ▶ Datos:

- Ejemplo:
 - ▶ Datos:
 - ★ Padres de Ana: María y José.

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- ▶ Usando la ecuación recursiva:

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- ▶ Usando la ecuación recursiva:

- ★ $H(\text{Ana}) = h(\text{Ana}) + 0,5 [H(\text{María}) + H(\text{José})]$

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- ▶ Usando la ecuación recursiva:

- ★ $H(\text{Ana}) = h(\text{Ana}) + 0,5 [H(\text{María}) + H(\text{José})]$
 - ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})]$

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- ▶ Usando la ecuación recursiva:

- ★ $H(\text{Ana}) = h(\text{Ana}) + 0,5 [H(\text{María}) + H(\text{José})]$
 - ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})]$
 - ★ $H(\text{José}) = h(\text{José}) + 0,5 [H(\text{Elena}) + H(\text{Pedro})]$

- Ejemplo:

- ▶ Datos:

- ★ Padres de Ana: María y José.
 - ★ Padres de María: Luisa y Carlos.
 - ★ Padres de José: Elena y Pedro.

- ▶ Usando la ecuación recursiva:

- ★ $H(\text{Ana}) = h(\text{Ana}) + 0,5 [H(\text{María}) + H(\text{José})]$
 - ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})]$
 - ★ $H(\text{José}) = h(\text{José}) + 0,5 [H(\text{Elena}) + H(\text{Pedro})]$

- ▶ Para los ancestros sin padres (Luisa, Carlos, Elena, Pedro):

$$H(p) = h(p)$$

- Ejemplo: (cont.)

- Ejemplo: (cont.)
 - ▶ Personas sin padres:

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$

$$H(\text{Carlos}) = 150$$

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$

- $H(\text{Carlos}) = 150$

- ★ $H(\text{Elena}) = 120$

- $H(\text{Pedro}) = 130$

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$

- $H(\text{Carlos}) = 150$

- ★ $H(\text{Elena}) = 120$

- $H(\text{Pedro}) = 130$

- ▶ Para sus hijos:

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$

- $H(\text{Carlos}) = 150$

- ★ $H(\text{Elena}) = 120$

- $H(\text{Pedro}) = 130$

- ▶ Para sus hijos:

- ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})] = 0 + 0,5 [100 + 150] = 125$

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$ $H(\text{Carlos}) = 150$

- ★ $H(\text{Elena}) = 120$ $H(\text{Pedro}) = 130$

- ▶ Para sus hijos:

- ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})] = 0 + 0,5 [100 + 150] = 125$

- ★ $H(\text{José}) = h(\text{José}) + 0,5 [H(\text{Elena}) + H(\text{Pedro})] = 0 + 0,5 [120 + 130] = 125$

- Ejemplo: (cont.)

- ▶ Personas sin padres:

- ★ $H(\text{Luisa}) = 100$ $H(\text{Carlos}) = 150$

- ★ $H(\text{Elena}) = 120$ $H(\text{Pedro}) = 130$

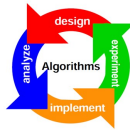
- ▶ Para sus hijos:

- ★ $H(\text{María}) = h(\text{María}) + 0,5 [H(\text{Luisa}) + H(\text{Carlos})] = 0 + 0,5 [100 + 150] = 125$

- ★ $H(\text{José}) = h(\text{José}) + 0,5 [H(\text{Elena}) + H(\text{Pedro})] = 0 + 0,5 [120 + 130] = 125$

- ★ $H(\text{Ana}) = h(\text{Ana}) + 0,5 [H(\text{María}) + H(\text{José})] = 0 + 0,5 [125 + 125] = 125$

Algoritmos recursivos



- 1 Recursividad
- 2 Algoritmo recursivo
 - Reparto de herencia
- 3 Problemas de naturaleza recursiva

Problemas de naturaleza recursiva

- Problemas Matemáticos

Problema	Solución
Potenciación	Cálculo de a^b usando divide y vencerás.
MCD	Algoritmo recursivo para el máximo común divisor.
Número de combinaciones	$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$

- Estructuras de Datos Recursivas

Problema	Solución
Recorrido en un árbol binario	Inorden, preorden, postorden.
Evaluar expresiones en árboles	Árboles de expresiones matemáticas.
Insertión y busca en BST	Árbol binario de búsqueda.
Altura o profundidad de árbol	Altura máxima desde la raíz.

- Problemas combinatorios

Problema	Solución
Problema de las N reinas	Colocar N reinas sin que se ataquen.
Suma de subconjuntos	Subconjuntos cuya suma sea un valor objetivo.
Coloración de grafos	Asignar colores sin conflictos.
Combinaciones posibles	Subconjuntos de tamaño k.

- Problemas sobre Cadenas

Problema	Solución
Palíndromos	Verificar si una cadena es palíndroma.
Reversión de cadenas	Invertir una cadena recursivamente.
Generación de subsecuencias	Todas las subsecuencias posibles de una cadena.
Conteo de ocurrencias	Contar cuántas veces aparece un patrón en una cadena.

- Inteligencia Artificial / Juegos

Problema	Solución
Minimax (juegos como ajedrez)	Algoritmo para juegos de dos jugadores con decisiones óptimas.
Árboles de decisión	Recorrido y evaluación de decisiones posibles.

- **Inteligencia Artificial / Juegos**

Problema	Solución
Minimax (juegos como ajedrez)	Algoritmo para juegos de dos jugadores con decisiones óptimas.
Árboles de decisión	Recorrido y evaluación de decisiones posibles.

- **Recursión en Lenguajes Naturales y Compiladores**

Problema	Solución
Análisis sintáctico	Uso de gramáticas recursivas para parsear código o lenguaje natural.
Evaluación de expresiones matemáticas	Recursión basada en reglas gramaticales.